SRL-2015 Memory Deep Analysis

SRL-2015-APT-ENTERPRISE — Memory-Injected Payload Deep Analysis

Case: SRL-2015-APT-ENTERPRISE **Run ID:** srl2015-deep-mem-20260610 **Agent:** deep-analysis (`manual.deep_re`) **Date:** 2026-06-10 **Classification:** Static reverse-engineering of live malware (read-only)

---

1. Executive Summary

Five Volatility `malfind` page dumps (8192 bytes each) recovered from the SRL-2015 enterprise intrusion were reverse-engineered statically. All five are the **same VB6-packed, self-injecting in-memory loader family**, captured as RWX (read/write/execute) injected pages inside processes masquerading as F-Response / `femc.exe` tooling.

Key conclusions:

- **One malware family, two compiler builds.** Multiple independent methods (ssdeep, position-wise byte identity, JMP-target bytes, and full i386 disassembly of the entry routine) confirm **two clusters** that execute the *identical* unpack algorithm with different register allocation — i.e. the same source recompiled. **Cluster A** = samples 17, 18 (EBP stack frame). **Cluster B** = samples 19, 20 (femc), 21 (nfury) (ESP-relative frame).
- **Technique: in-memory unpack / self-injection — not a download stub, not API-hash shellcode.** An 8-byte length header points at an `E9` JMP loader stub that `VirtualAlloc`s a buffer, runs an embedded **LZMA-family range decoder** to inflate a compressed second stage, and resolves imports by **plaintext name** (`OpenProcess` / `VirtualProtect` / `GetModuleHandleA`). The loader carries RWX-flip (`VirtualProtect`) and cross-process (`OpenProcess`) capability.
- **No C2 / config recoverable from these dumps.** A 256-key single-byte XOR + base64 sweep found **0 readable network indicators**; payload-tail entropy (~3.97 bits/byte, 48.6% zero bytes) rules out an embedded encrypted config. The real C2 lives in the LZMA-compressed second stage, which is not present in these page captures.
- **Detection delivered.** YARA rule `SRL2015_MemInject_VB_Family` fires on all five samples and on any future recompile (anchored on the header+JMP stub, `msvbvm`, and the injection API cluster).

---

2. Methodology — Static-Only

> **STATIC ANALYSIS ONLY. No sample was executed.** These are live malware payloads.

1. **Acquisition.** The 5 `*malfind*.bin` entries were extracted read-only from the password-protected quarantine zip (`quarantine/srl2015-samples.zip`, password `infected`) into a `0700` tmpfs working directory. After analysis the temp was wiped; only the encrypted zip persists.
2. **Hashing.** SHA-256 computed per sample (Section 9).
3. **Disassembly.** `objdump` in binary mode (`-b binary -m i386 -M intel`) plus capstone via the project venv (`/home/admin2/agentropix-sift/.venv/bin/python`). Full disasm saved to `disasm-variantA.txt` / `disasm-variantB.txt`. **All disassembly is real tool output.**
4. **Strings.** ASCII (>=4) and UTF-16LE; saved to `strings-all.txt`.
5. **Clustering.** ssdeep fuzzy hashing + position-wise byte-identity comparison of the fixed 8192-byte files + JMP-target byte discrimination.
6. **IOC / config hunt.** Single-byte XOR brute (keys `0x00`–`0xFF`) × base64 detection with a strict network-indicator matcher; payload-tail entropy/zero-byte analysis.
7. **Detection.** YARA rule authored and compiled (`yara`, exit 0).

---

3. Bitness & Header

Bitness: 32-bit x86 (i386). Confirmed by coherent i386 decode and stdcall epilogues with immediate-pop returns (`ret 0xC` / `ret 0x10` / `ret 0x18`) and 32-bit operands/addressing throughout. Sample 21 (the Win7-64 nfury host) is a **32-bit (WOW64) injected page**, not native x64.

Header (first 8 bytes): `08 00 00 00 00 00 00 00` — an 8-byte little-endian prefix = `0x08`, the **offset at which the entry/JMP stub begins**. Code does **not** start at offset 0; it starts at offset `0x08`. The byte immediately after the JMP (offset `0x0D`) is a separate worker function — the decompressor.

---

4. Per-Variant Disassembly Walkthrough

4.1 Variant A (samples 17, 18) — EBP stack frame, `jmp 0x804`

```
00000008  e9 f7 07 00 00     jmp    0x804          ; -> real entry
routine
0000000d  55                 push   ebp            ; decoder worker
prologue
0000000e  8b ec              mov    ebp,esp
00000010  83 ec 34           sub    esp,0x34
00000013  8b 45 08           mov    eax,DWORD PTR [ebp+0x8]   ; arg =
API/ctx struct
00000016  8b 48 08           mov    ecx,DWORD PTR [eax+0x8]   ; field
+0x8
...
```

```
00000026  8b 70 0c           mov    esi,DWORD PTR [eax+0xc]   ; field
+0xc
0000002c  d3 e3              shl    ebx,cl                    ; SHL by
cl
0000003c  b8 00 03 00 00     mov    eax,0x300                 ; 0x300
...
```

Entry at `0x804` uses a classic EBP frame (`55 8B EC` push ebp/mov ebp,esp; `83 EC 34`).

4.2 Variant B (samples 19, 20-femc, 21-nfury) — ESP-relative frame, `jmp 0x86D`

```
00000008  e9 60 08 00 00     jmp    0x86d          ; -> real entry
routine
0000000d  83 ec 30           sub    esp,0x30       ; ESP-relative
frame (no push ebp)
00000010  8b 44 24 34        mov    eax,DWORD PTR [esp+0x34]  ; arg =
API/ctx struct
00000014  8b 48 08           mov    ecx,DWORD PTR [eax+0x8]   ; field
+0x8  (same as A)
0000001a  be 01 00 00 00     mov    esi,0x1
00000021  d3 e3              shl    ebx,cl                    ; SHL by
cl
00000036  b8 00 03 00 00     mov    eax,0x300                 ; 0x300
...
00000043  05 36 07 00 00     add    eax,0x736                 ; +0x736
```

Same algorithm, different codegen. Both variants read the same struct fields (`[arg+0x4]`, `[arg+0x8]`, `[arg+0xC]`), shift by `cl`, load `0x300`, shift, and add `0x736`. The only difference is A's EBP frame vs B's ESP-relative frame — the hallmark of one source recompiled with a different optimization/register-allocation pass. This is the cleanest attribution evidence that A and B share an author.

---

5. Injection / Unpack Chain

The loader is an **in-memory unpack / self-injecting stub** — **not** a remote download stub and **not** API-hash shellcode.

1. **Header → JMP.** 8-byte LE prefix (`0x08`) → at offset `0x08` an `E9 rel32` JMP to the entry routine (A: `0x804`, B: `0x86D`).
2. **Size calc.** Entry reads header `byte[hdr+4]`, does `idiv 9`, `idiv 5`, then computes `((0x300 << cl) + 0x736) << 4` for the buffer size.
3. **Allocate.** `VirtualAlloc(NULL, size, MEM_COMMIT 0x1000, PAGE_READWRITE 0x04)` — `VirtualAlloc`/`VirtualFree` are passed in as pre-resolved kernel32 pointers via a struct (`[ptr+0x08]=VirtualAlloc`, `[ptr+0x0C]=VirtualFree`).
4. **Decompress.** Calls the worker at offset `0x0D`, an **LZMA-family range decoder** (signatures: `shr esi,0xB`, kTopValue `0x800`, probability update `shr 5`, range-renorm threshold `0x1000000`, prob tables via `imul esi,esi,0xC00` / `+0x1CD8`), inflating an embedded compressed blob into the buffer.

5. **Release / return.** `VirtualFree(buf, 0, MEM_RELEASE 0x8000)`, returning the decoded buffer to the caller.

Import resolution is by **plaintext name**, not API hashing. A PIC import resolver at file offset `0x10D8` self-locates (`call $+5; pop ebx; sub ebx,<delta>`), walks a plaintext import-name list (`kernel32`/`user32` + `OpenProcess`, `VirtualProtect`, `GetModuleHandleA`, `ExitProcess`, `CloseHandle`, `MessageBoxA`, `wsprintfA`), supports ordinal imports (`test eax,0x80000000` = `IMAGE_ORDINAL_FLAG`), and writes resolved addresses into an IAT. VB6 runtime-error format strings are present (`"The procedure %s could not be located in the DLL %s."`, `"The ordinal %d could not be located in the DLL %s."`). `msvbvm` (VB6 runtime, `msvbvm60`) is referenced — a **VB6-packed loader stub**, the classic shape Volatility `malfind` flags.

Capabilities surfaced by imports: `VirtualProtect` = RWX flip (self-modifying execution); `OpenProcess` = cross-process injection capability.

The second stage is **LZMA-compressed**; there is **no plaintext MZ/PE** in the dumped page. The real payload (and its C2) only materializes after decompression, which these page captures do not contain.

---

6. Cluster Analysis

Cluster	Members	JMP stub	Entry	Frame	Hosts
A	17 (pid 23476), 18 (pid 26340)	`E9 F7 07 00 00` → `0x804`	`0x804`	EBP (`55 8B EC`; `83 EC 34`)	win2008R2 controller
B	19 (pid 145896), 20 (pid 151132 femc), 21 (nfury pid 328)	`E9 60 08 00 00` → `0x86D`	`0x86D`	ESP-rel (`83 EC 30`; `8B 44 24 34`)	win2008R2 controller + win7-64 nfury (WOW64)

Quantitative agreement (multiple independent methods):

- **ssdeep fuzzy match:** intra-A 17 ↔ 18 = **94**; intra-B 19 ↔ 20 = **96**, 19 ↔ 21 = **97**, 20 ↔ 21 = **91**. Every cross-cluster A-vs-B pair = **52–58**.
- **Position-wise byte identity (8192-byte files):** 17 vs 18 = **99.1%**; within B 19 vs 21 = **99.8%**, 19 vs 20 = **98.8%**, 20 vs 21 = **98.7%**. Every A-vs-B pair = **36.9%**.
- **JMP target** is the cleanest discriminator (A `+0x7F7` vs B `+0x860`).
- **Disassembly** confirms the EBP-vs-ESP frame split while both run the identical size/unpack math.

Within-cluster byte differences are scattered runtime-patched data (handles/addresses/cookies) in the post-code memory-manager region (`>=0x900`), **not** a configuration string.

---

7. IOCs

7.1 Negative result — no C2 / config in these dumps

- **256-key XOR + base64 sweep** across all 5 payloads with a strict network-indicator matcher (`http(s)://`, domains w/ common TLDs, IPv4+port, User-Agent, `Mozilla/`, `cmd.exe`, `powershell`, `\pipe\`, `Global\` mutex) → **0 readable network indicators** under any key. A looser regex produced only XOR-garbage false positives resolving to no host/IP/URL.
- **No embedded config block.** Payload-tail (`>=0x800`) entropy ≈ **3.97 bits/byte** with **48.6% zero bytes** — far too low for an encrypted/packed config. The post-code region is captured Windows memory-manager metadata (repeating cookie `c47e e7d5`, addresses `00a00c00`/`00900b00`, `0000ffff` markers) — VAD/heap structures snapshotted by `malfind`.
- **No dropper cross-references.** No `usboesrv.exe` / `a.exe` / `spinlock.exe` / `svchost.exe` / `femc` / `f-response` / `nfury` strings inside payload bytes; the host/process link is contextual (Volatility dump labels) only.

7.2 Actionable IOCs

- **5 × SHA-256** of the malfind payloads (Section 9).
- **`msvbvm60` VB6 injected-RWX-page signature** with the `OpenProcess`/`GetModuleHandleA`/ `VirtualProtect` plaintext API cluster.
- **YARA rule `SRL2015_MemInject_VB_Family`** (Section 8).
- **Variant JMP-stub bytes:** A = `08 00 00 00 00 00 00 00 E9 F7 07 00 00`; B = `08 00 00 00 00 00 00 00 E9 60 08 00 00`.

---

8. YARA Rule

Rule `SRL2015_MemInject_VB_Family` (file: `srl2015_meminject.yar`, compiles clean, `yara` exit 0). It fires on **both variants and any future recompile** — anchored on the 8-byte length header + `E9` JMP stub at offset 0, `msvbvm`, the `kernel32` import-resolver target, the `OpenProcess`/`GetModuleHandleA`/`VirtualProtect` injection API cluster, and a VB runtime-error artifact or the contiguous `OpenProcess\x00GetModuleHandleA\x00VirtualProtect` API table; bounded `filesize < 256KB` for fast malfind scanning. Variant discriminators (`$jmp_A` `0x804` / `$jmp_B` `0x86D`) are retained for cluster attribution.

Full rule: `/home/admin2/agentropix-sift/Reports_results/SRL2015-DELIVERABLE/deep-analysis/srl2015_meminject.yar`

---

9. Hash-Verified Samples

#	File	PID	Process	Host	Variant	
17	`17_..._pid23476.bin`	23476	f-response-lm-	win2008R2-controller	A	4

#	File	PID	Process	Host	Variant	
18	`18_..._pid26340.bin`	26340	f-response-lm-	win2008R2-controller	A	7
19	`19_..._pid145896.bin`	145896	f-response-ent	win2008R2-controller	B	e
20	`20_..._pid151132_femc.bin`	151132	femc.exe	win2008R2-controller	B	d
21	`21_..._pid328.bin`	328	f-response-ent	win7-64-nfury (WOW64)	B	a

10. Attribution

- **Single malware family, single author, two compiler builds.** The identical unpack algorithm (same struct-field reads, `SHL cl`, `0x300`/`0x736` constants, LZMA range decoder, plaintext-name import resolver) across an EBP-frame build (A) and an ESP-frame build (B), combined with the hard ssdeep/byte-identity cluster boundary (intra ≥91 / 98.7%+, cross 52–58 / 36.9%), is consistent with one codebase recompiled, not two unrelated tools.
- **Enterprise-wide deployment.** Cluster B spans both the Win2008R2 controllers and the Win7-64 nfury host (as a WOW64 injection), evidencing a single loader pushed across multiple hosts/OSes.
- **Tradecraft is mid-tier, not nation-state-novel.** Plaintext-name API resolution (no hashing), VB6 packing, and `msvbvm60` artifacts indicate a commodity/older packer style rather than bespoke position-independent shellcode. Process masquerade as F-Response / `femc.exe` (legitimate IR tooling) is the notable OPSEC choice — blending into the responders' own toolset.
- **Caveat.** Final-stage attribution (C2 infrastructure, campaign linkage) requires the LZMA-decompressed second stage, which is absent from these 8KB malfind page captures.

11. MITRE ATT&CK Mapping

Technique	ID	Where observed
Process Injection	T1055	OpenProcess import + RWX malfind page
Process Injection: Process Hollowing / Reflective	T1055.012	In-memory self-injecting loader
Reflective Code Loading	T1620	VirtualAlloc → LZMA decode → execute in memory
Software Packing	T1027.002	VB6/`msvbvm60` packer + LZMA-compressed second stage

Technique	ID	Where observed
Deobfuscate/Decode Files or Information	T1140	Embedded LZMA range decoder at offset 0x0D
Masquerading	T1036	`femc.exe` / F-Response process masquerade
Native API	T1106	VirtualAlloc/VirtualFree/VirtualProtect/OpenProces

Generated by Agentropix-SIFT deep-analysis agent · static reverse engineering · no execution. Artifacts: `deep-findings.json`, `disasm-variantA.txt`, `disasm-variantB.txt`, `strings-all.txt`, `ioc-config-summary.txt`, `INJECTION-ANALYSIS.md`, `srl2015_meminject.yar`.